

Servicio MLOps end-to-end: predicción de precios de vivienda

[COMPLETAR] Curso · Sección · Segundo semestre 2026 | Integrantes: [NOMBRE 1], [NOMBRE 2], [NOMBRE 3], [NOMBRE 4]

1. Problema

Se aborda un problema de regresión: predecir el precio de venta (SalePrice, en USD) de una vivienda a partir de un subconjunto reducido de variables numéricas del inmueble. El objetivo del trabajo no es maximizar la sofisticación del modelo, sino construir un sistema completo y operable: entrenamiento reproducible, contrato de API validado, contenedor portable y un pipeline de integración y despliegue continuo.

2. Datos

Se utilizó el dataset público **Ames Housing** (competencia Kaggle "House Prices: Advanced Regression Techniques"), con 1460 registros y 81 columnas originales. De estas, se descartaron deliberadamente las ~70 columnas categóricas para mantener el pipeline simple y 100% numérico, y se seleccionaron las 6 columnas numéricas sin valores nulos con mayor correlación lineal respecto al precio de venta:

Variable	Descripción	Corr. con precio
overall_qual	Calidad general de materiales y acabados (1-10)	0.79
gr_liv_area	Superficie habitable sobre el suelo (sqft)	0.71
garage_cars	Capacidad del garaje (autos)	0.64
total_bsmt_sf	Superficie total del sótano (sqft)	0.61
full_bath	Baños completos sobre el suelo	0.56
year_built	Año de construcción original	0.52

El dataset crudo se versiona en `data/housing_train_raw.csv` dentro del repositorio.

3. Modelo y entrenamiento

Se entrenó un pipeline de scikit-learn compuesto por un `StandardScaler` seguido de un `RandomForestRegressor` (200 árboles, profundidad máxima 12). Se usó una semilla fija (`random_state=42`) y una separación explícita 80/20 entre entrenamiento y prueba, evaluando las métricas únicamente sobre el conjunto de prueba (datos no vistos durante el ajuste).

Métrica	Valor	Interpretación
R ²	0.888	El modelo explica ~89% de la varianza del precio
RMSE	\$29,374	Error cuadrático medio sobre el conjunto de prueba
MAE	\$19,207	Error absoluto promedio (~10-11% del precio medio, ~\$180k)

Los artefactos (`model.pkl` y `metadata.json` con features, fecha de entrenamiento, semilla y métricas) se generan de forma reproducible mediante `src/train.py`, y se regeneran automáticamente en cada build de la imagen Docker.

4. Decisiones de diseño

[COMPLETAR — el equipo debe describir aquí, con sus propias palabras, decisiones concretas y el porqué. Ejemplos de preguntas que la defensa oral puede hacer sobre esta sección:]

- ¿Por qué RandomForest y no un modelo lineal? ¿Se probó alguna alternativa?
- ¿Por qué se descartaron las columnas categóricas en vez de codificarlas?
- ¿Por qué se entrena el modelo dentro del build de Docker y no se versiona el .pkl directamente?
- ¿Qué llevó a fijar los rangos de validación de Pydantic (ej. overall_qual entre 1 y 10)?
- ¿Por qué se eligió Render sobre otras opciones de despliegue?

5. Resultados

[COMPLETAR — capturas de pantalla o resumen de: la ejecución verde del pipeline en la pestaña Actions, el servicio respondiendo en /health desde internet, y un ejemplo de /predict con salida real. Comentar si el error del modelo (MAE ~\$19k) es aceptable para el caso de uso planteado.]

6. Limitaciones conocidas y trabajo futuro

- Se descartaron ~70 columnas categóricas (ej. Neighborhood, ExterQual) que aportarían señal significativa en un modelo de producción real; con más tiempo se agregaría un encoder dentro del mismo Pipeline de sklearn.
- No hay monitoreo de data drift ni reentrenamiento automático.
- No hay autenticación en la API (pensado para uso académico/demo).
- El modelo no reporta intervalos de confianza por predicción.
- No hay versionado formal de datasets (ej. DVC).
- El plan gratuito de Render duerme el servicio tras inactividad; el primer request post-inactividad puede tardar 30-50s.

7. Distribución del trabajo

[COMPLETAR — cada integrante debe responder por su parte en la defensa oral]

Integrante	Módulos / responsabilidades
[Nombre 1]	[ej. entrenamiento del modelo, selección de features]
[Nombre 2]	[ej. API FastAPI, contratos Pydantic]
[Nombre 3]	[ej. Docker, docker-compose, CI/CD]
[Nombre 4]	[ej. despliegue en Render, documentación, video]

Nota: este documento fue generado como plantilla inicial con asistencia de IA (Claude, Anthropic) a partir del código y las métricas reales del proyecto. Las secciones marcadas [COMPLETAR] requieren el análisis y la redacción propia del equipo antes de la entrega.